Shailesh Yadav-da

Data Analyst

Matplotlib — Complete Course to Transform Your Data Skills

matplotlib

Welcome To My Post

Scroll More

Matplotlib Library — Python Data Visualization

A Complete Beginner to Data Analyst Guide

What is Matplotlib?

Matplotlib is a Python library used to create graphs and charts.
It helps convert raw data into visual insights.

Why Matplotlib is Important?

- Makes data easy to understand
- Used in EDA (Exploratory Data Analysis)
- Widely used by Data Analysts
- Base library for advanced visualization tools

Where is Matplotlib Used?

- Sales Analysis
- Business Reports
- Dashboards
- Data Analysis Projects
- Interview Case Studies

What You Will Learn in This Guide

Line Charts
Bar Charts
Histogram
Scatter Plot
Labels, Titles & Styling
Real Business Use Cases

BAR PLOT (Bar Chart)

What is a Bar Plot?

A Bar Plot is a chart that uses rectangular bars to show comparison between

categories.

Each bar represents:

- a category (Region, Product, Category, etc.)
- a numerical value (Sales, Profit, Quantity)

Taller bar = Higher value

Why Bar Plot is Important?

Bar plots are used when we want to:

- Compare values across different groups
- Identify highest and lowest performers
- Make business decisions easily

One of the most used charts in Data Analysis.

When Should We Use a Bar Plot?

Use a Bar Plot when:

- Data is categorical
- You want comparison, not trend

Best Examples:

- Region-wise sales
- Category-wise profit
- Product-wise quantity sold

Real-Life / Business Use

- Which region has the highest sales?
- Which product category performs best?
- Which department needs improvement?

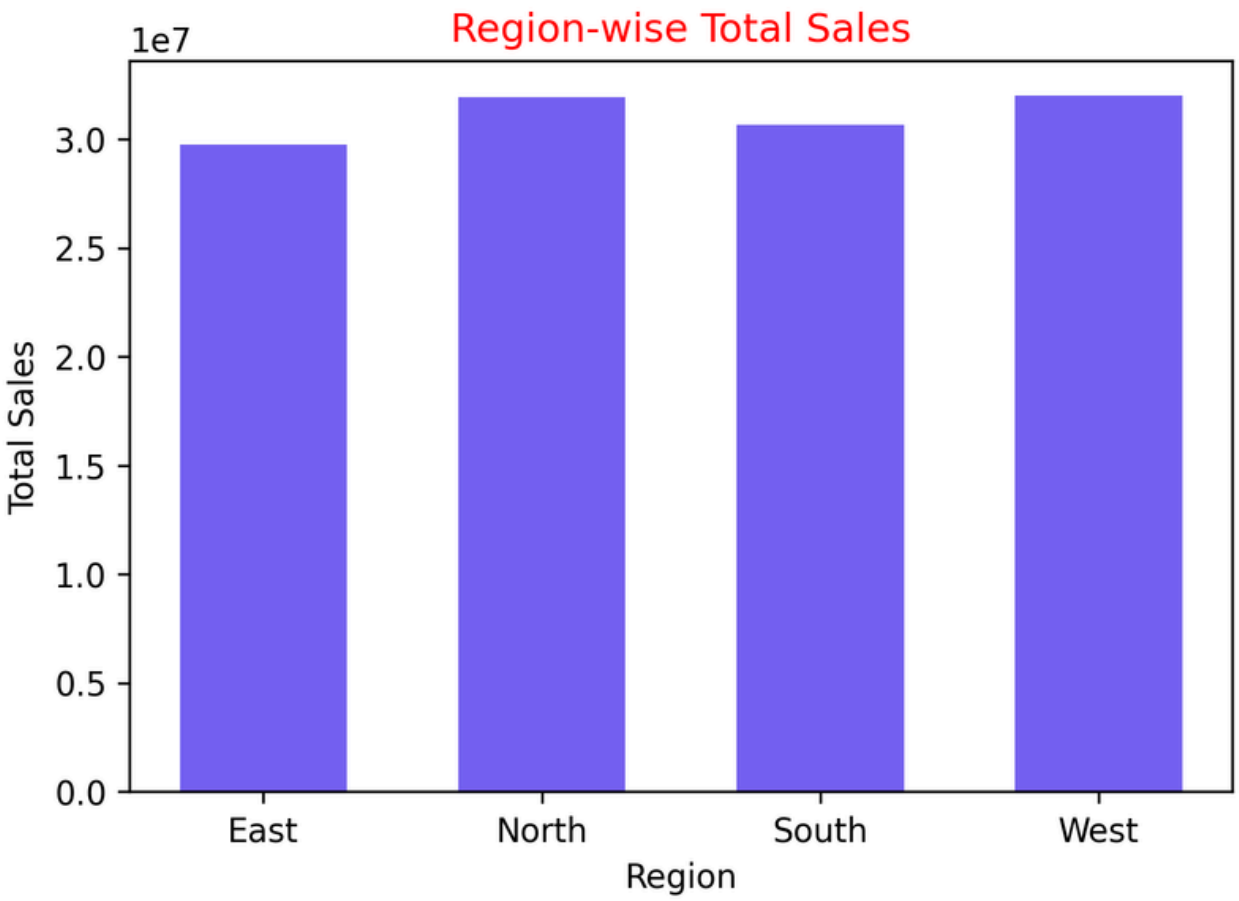
Managers and stakeholders love bar charts because they are easy to understand.

Step 1: Group data

```
region_sales = df.groupby('region')
['sales_amount'].sum()


plt.figure(figsize=(6, 4))
plt.bar(region_sales.index, region_sales.values,
width=0.6, color='#7662f0',)
plt.xlabel("Region",fontsize=10)
plt.ylabel("Total Sales",fontsize=10)
plt.title("Region-wise Total Sales",color="r")
plt.savefig("region_wise_sales.png", dpi=300,
bbox_inches='tight')
plt.show()
```

🎯 Important Save Parameters (Easy Table)

Parameter	Use
`dpi=300`	High quality image
`bbox_inches='tight'`	Extra white space remove
`.png`	Best for Canva & LinkedIn
`.jpg`	Smaller size
`.pdf`	Reports

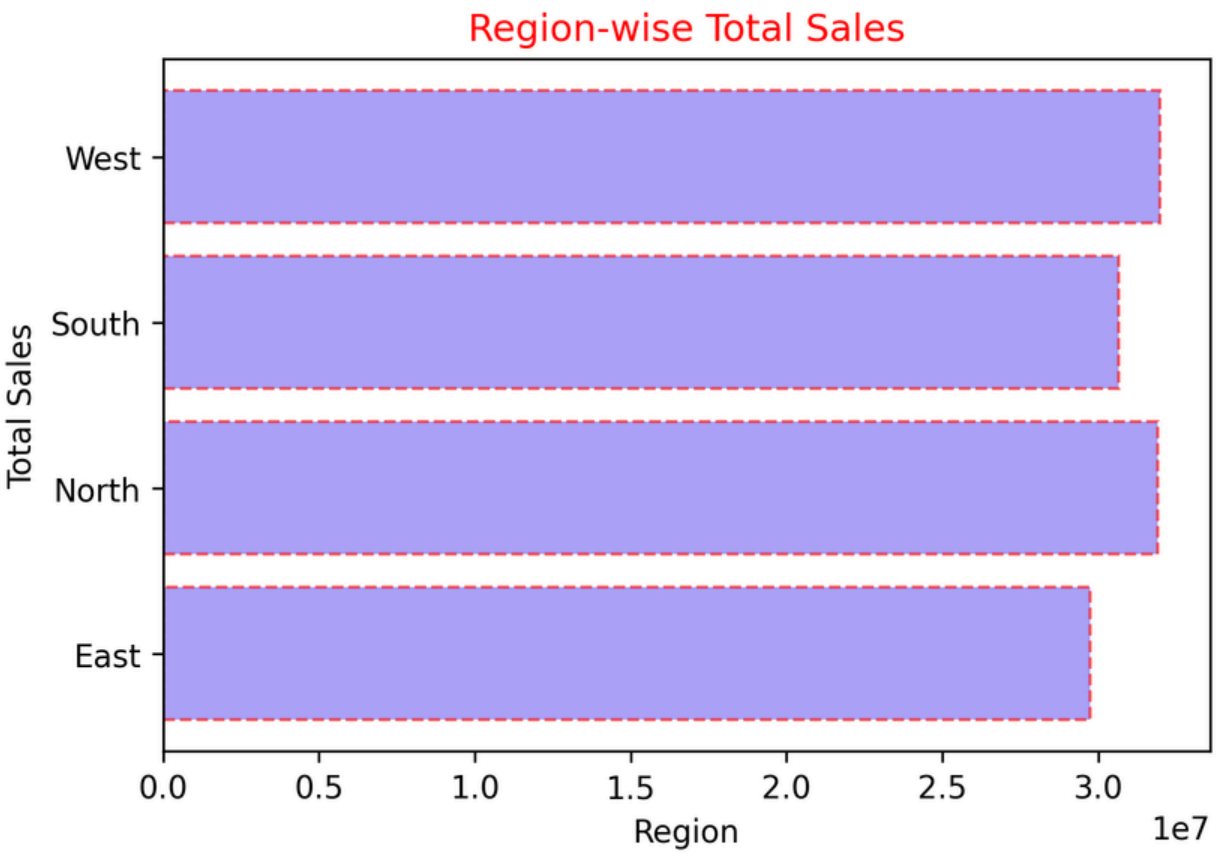


Advance charts:

Matplotlib Chart Customization

- figsize → controls chart size
- color → bar fill color
- edgecolor → bar border color
- linewidth → border thickness
- linestyle → border style
- alpha → transparency
- xlabel, ylabel → axis labels
- title → chart heading

```
plt.figure(figsize=(6, 4))

plt.barh(
    region_sales.index,
    region_sales.values,
    color='#7662f0',
    edgecolor="r",
    linewidth=1,
    linestyle="--",
    alpha=0.6
)

plt.xlabel("Region", fontsize=10)
plt.ylabel("Total Sales", fontsize=10)
plt.title("Region-wise Total Sales", color="r")

plt.show()
```


Region-wise Total Sales
West
South
North
East
Total Sales
0.0
0.5
1.0
1.5
2.0
2.5
3.0
1e7
Region

Scatter Plot

What is a Scatter Plot?

A scatter plot displays data points as dots to show the relationship between two numerical variables.

Why / When do we use a Scatter Plot?

Use a scatter plot when you want to:

- Analyze relationship between two numeric variables
- Check correlation (positive, negative, none)
- Identify outliers
- Support business decisions using data

Typical questions it answers:

- Does X affect Y?
- Do values increase together?

Data

```
simple_df = df.head(50)


plt.figure(figsize=(6, 4))

plt.scatter(
    simple_df["Quantity"],
    simple_df["Sales_Amount"],
    alpha=0.7,
    color="y",
    s= 80
)
plt.xlabel("Quantity",fontsize=10)
plt.ylabel("Sales Amount",fontsize=10)
plt.title("Quantity vs Sales Amount ")
plt.show()
```

Output

Advance:

Popular Color Maps (MOST USED)

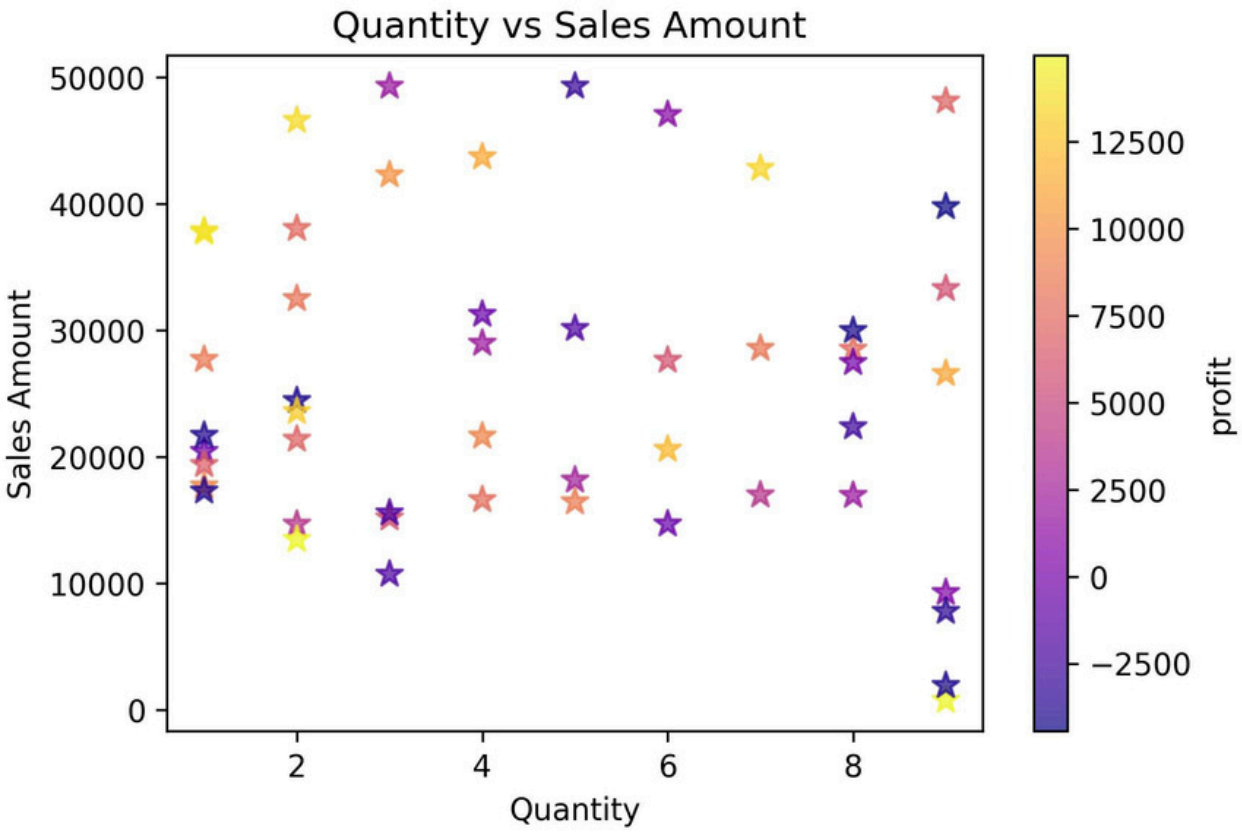
Sequential (Low → High values)

Best for continuous data

- "viridis" (MOST RECOMMENDED)
- "plasma"
- "inferno"
- "magma"
- "cividis"

Use when: Profit, Sales, Quantity, Price

```
plt.figure(figsize=(6, 4))

plt.scatter(
    simple_df["Quantity"],
    simple_df["Sales_Amount"],
    c=simple_df["Profit"],
    cmap="plasma",
    alpha=0.7,
    s= 80,
    marker="*"
)
plt.colorbar(label="profit")
plt.xlabel("Quantity",fontsize=10)
plt.ylabel("Sales Amount",fontsize=10)
plt.title("Quantity vs Sales Amount ")
plt.show()
```


Quantity vs Sales Amount
Sales Amount
50000
40000
30000
20000
10000
0
2
4
6
8
Quantity
12500
10000
7500
5000
2500
0
−2500
profit

Histogram

What is a Histogram?

A histogram is a chart used to show the distribution of a single numerical variable. It groups values into ranges (bins) and shows how frequently values fall into each range.

Why / When do we use a Histogram?

Use a histogram when you want to:

- Understand data distribution
- See where most values lie
- Identify skewness (left / right)
- Detect outliers

Typical questions:

- Most sales values kis range me aate hain?
- Profit zyada tar positive hai ya negative?

Data Requirement

- Only ONE numeric column is required

Perfect for continuous data like:
Sales, Profit, Quantity, Price

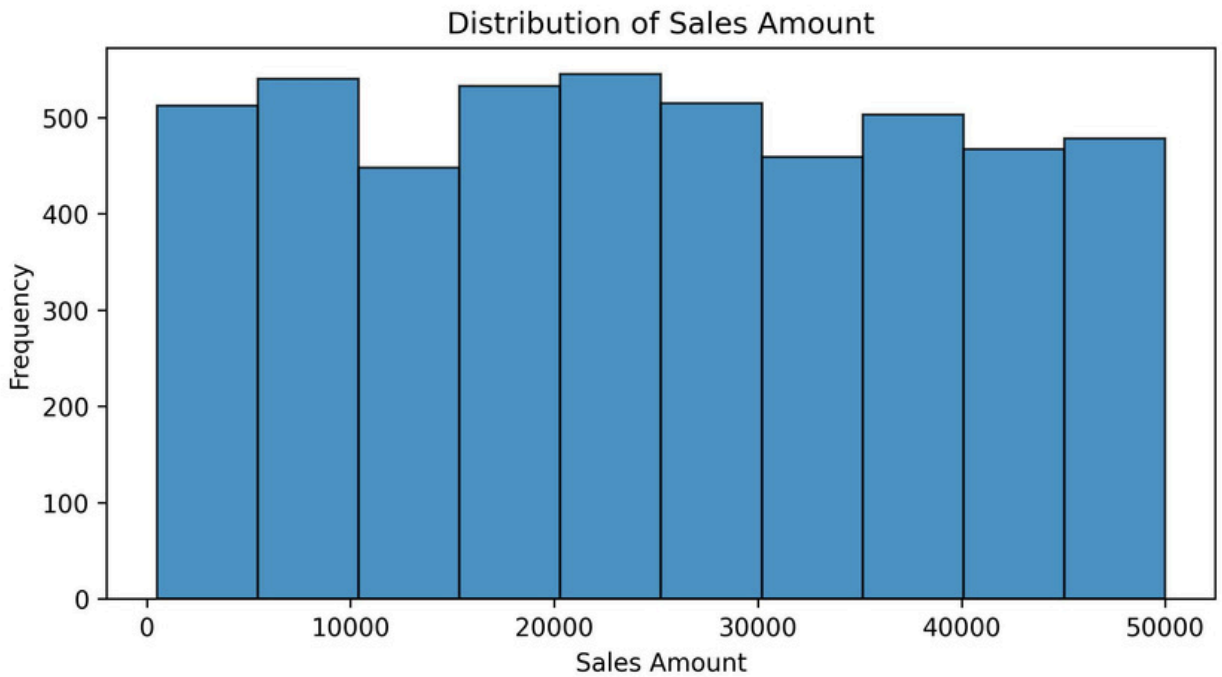
Data Selection (Simple & Clean)

```
sales_data = df['Sales_Amount']
```

No groupby No aggregation

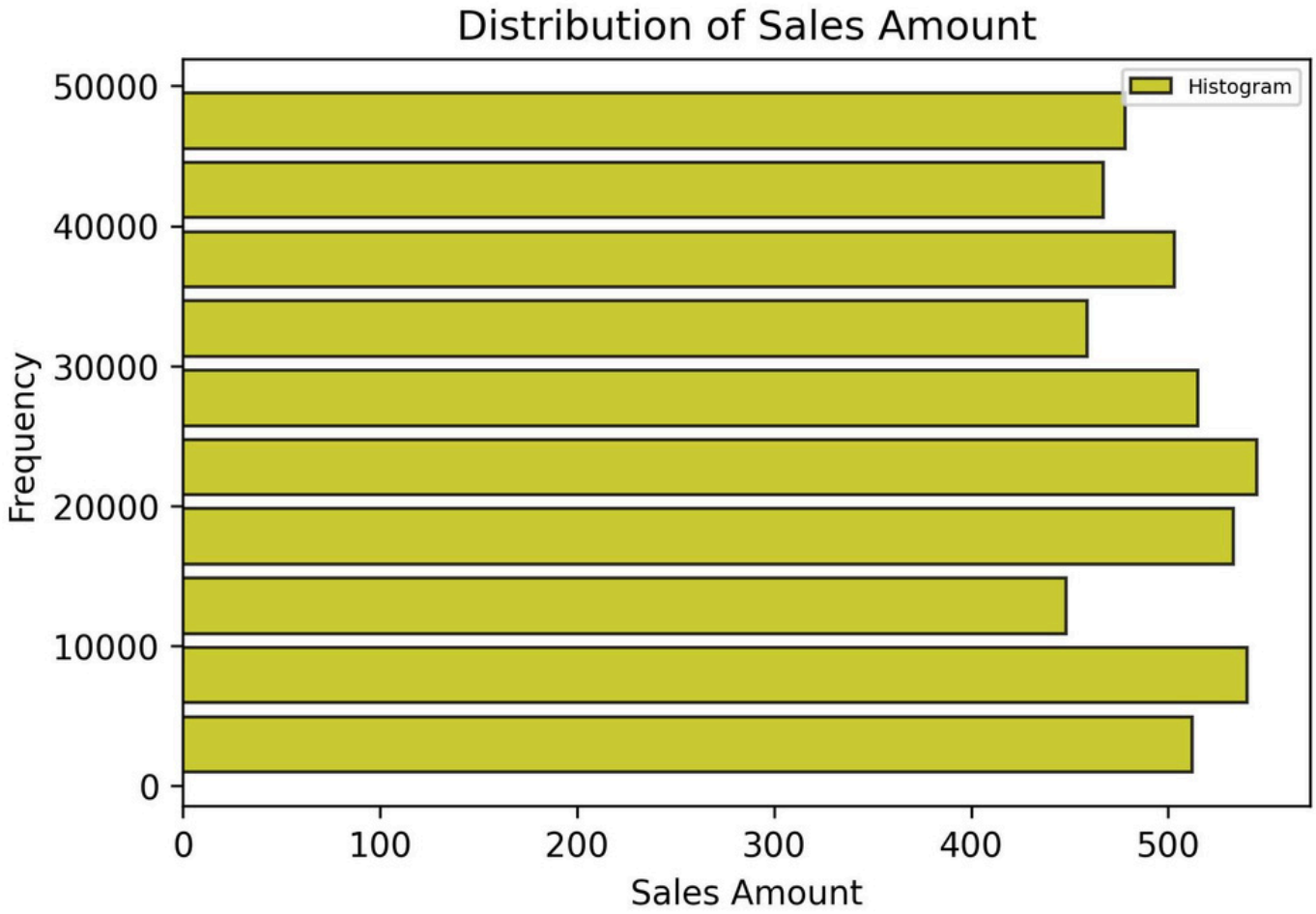
```
plt.figure(figsize=(8, 4))
plt.hist(
sales_data,
bins=10,
alpha=0.8,
edgecolor="black",
)
plt.xlabel('Sales Amount')
plt.ylabel('Frequency')
plt.title('Distribution of Sales Amount')
plt.show()
```

Output:



Advance:

```
plt.figure(figsize=(6, 4))
plt.hist(
sales_data,
bins=10,
alpha=0.8,
edgecolor="black",
rwidth=0.8,
color='y', orientation='horizontal',
label = 'Histogram'
)
plt.xlabel('Sales Amount')
plt.ylabel('Frequency')
plt.title('Distribution of Sales Amount')
plt.legend(loc=1,fontsize= 6)
plt.show()
```


Distribution of Sales Amount
Histogram
Frequency
50000
40000
30000
20000
10000
0
0
100
200
300
400
500
Sales Amount

Pie Plot:

What is a Pie Plot?

A Pie Plot is used to show the proportional contribution (percentage share) of different categories in a dataset.

It represents data in a circular form where:

- Each slice = category
- Slice size = percentage contribution

Why / When do we use a Pie Plot?

Use a pie chart when you want to:

- Show percentage distribution
- Compare part-to-whole relationship
- Present simple category contribution

Typical business questions:

- Which industry contributes most to total sales?
- Which region generates the highest share of revenue?

Use only when categories are limited (3–6 best).

Data Requirement

Pie chart needs:

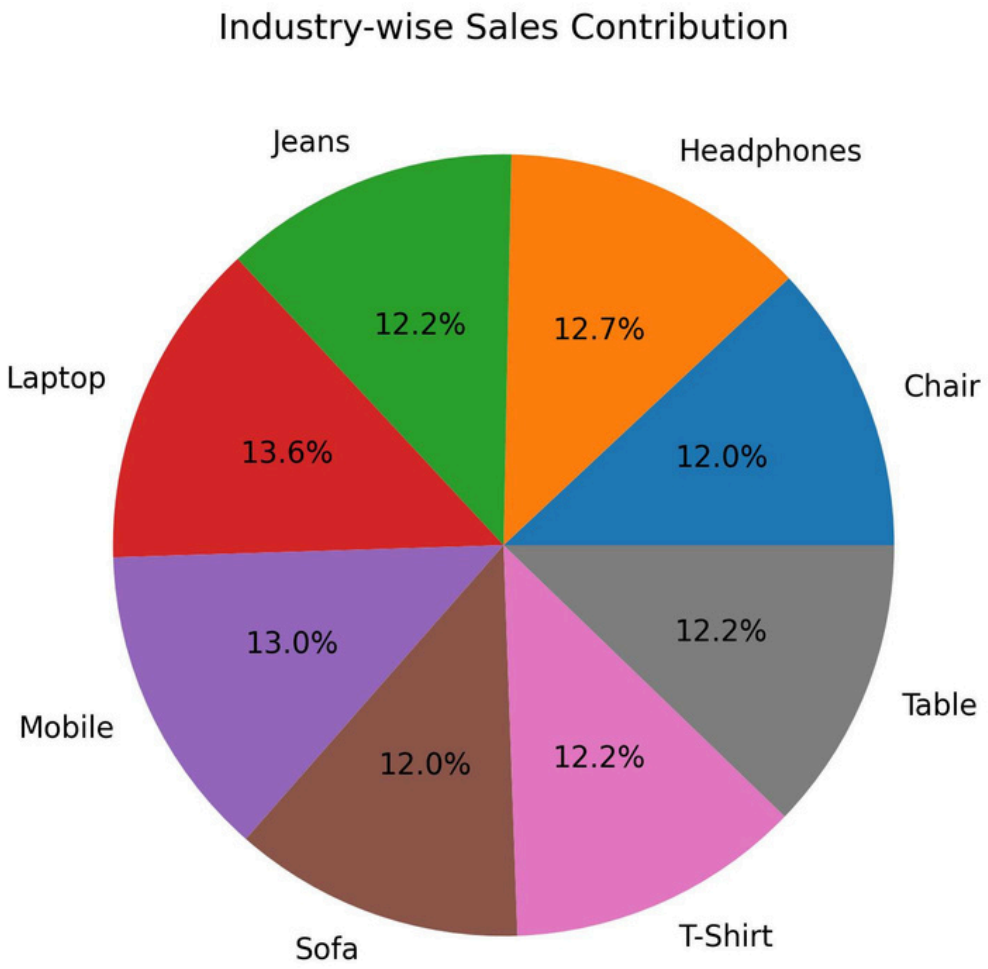
- One categorical column
- One numeric column (aggregated)

Data Preparation

```
industry_sales = df.groupby('Product')['Sales_Amount'].sum()
```

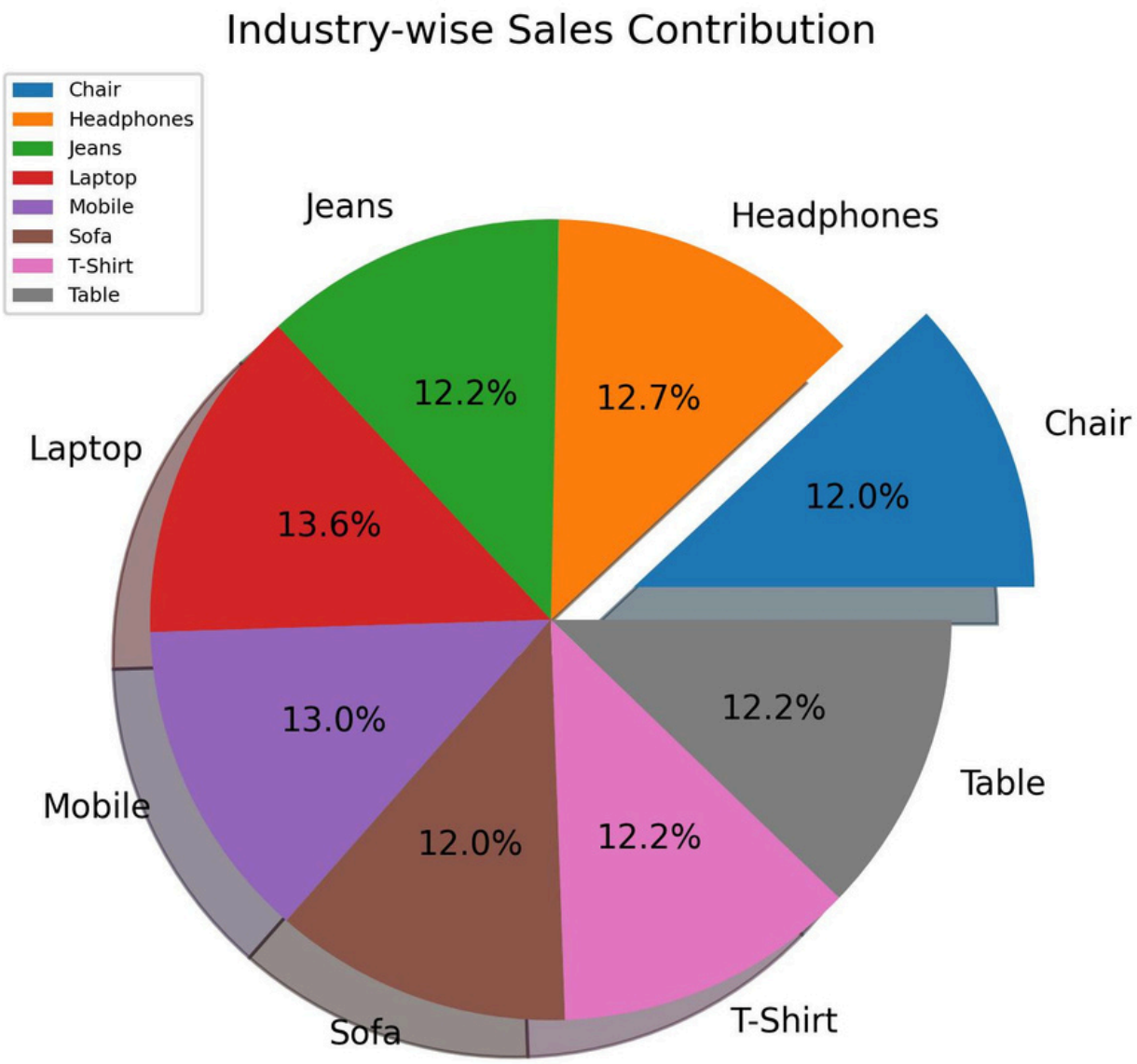
How to Create a Pie Plot (Matplotlib)

```
plt.figure(figsize=(6, 6))
plt.pie(
industry_sales,
labels=industry_sales.index,
autopct='%1.1f%%',
)
plt.title('Industry-wise Sales Contribution')
plt.show()
```



Advance plot:

```
plt.figure(figsize=(6, 6))
ex = [0.2, 0, 0, 0,0,0,0,0]
plt.pie(
industry_sales,
labels=industry_sales.index,
explode=ex,
autopct='%1.1f%%',
shadow=True,
radius=0.9,
labeldistance= 1.1,
startangle=0,
)
plt.title('Industry-wise Sales Contribution')
plt.legend(loc=2, fontsize=6)
plt.savefig("pieADV.jpg", dpi=300,bbox_inches='tight')
plt.show()
```


Industry-wise Sales Contribution
Chair
Headphones
Jeans
Laptop
Mobile
Sofa
T-Shirt
Table
Jeans
Headphones
Chair
Laptop
Mobile
Table
Sofa
T-Shirt
12.2%
12.7%
12.0%
13.6%
13.0%
12.2%
12.0%
12.2%

Stem Plot:

What is a Stem Plot?

A Stem Plot is used to visualize discrete data points where each value is

represented by:

- A vertical line (stem)
- A marker at the top (data point)

It is similar to a line plot but does NOT connect points with a continuous line.

Why / When do we use a Stem Plot?

Use a stem plot when:

- Data is discrete (not continuous)
- You want to highlight individual observations
- You are analyzing sequences or indexed values

Typical use cases:

- Monthly profit values
- Daily sales counts
- Signal processing data
- Comparing exact point differences

Data Requirement

- X values (index / sequence / time)
- Y values (numeric values)

Best for small to medium datasets.

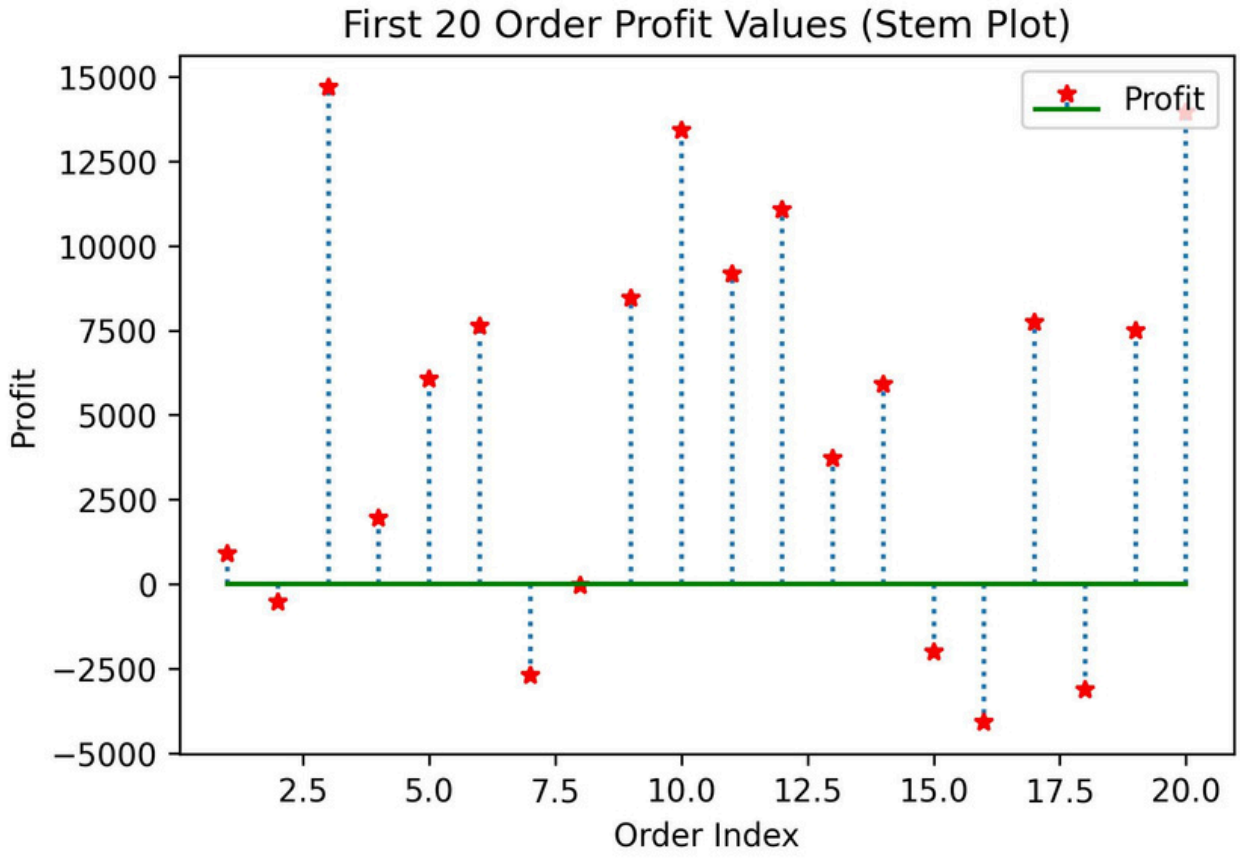
Data Selection

```
profit_sample = df['Profit'].head(20)
```

How to Create Stem Plot

```
plt.figure(figsize=(6,4))
plt.stem(
profit_sample.index,
profit_sample.values,
linefmt=':',
markerfmt='r*',
basefmt='g',
label = 'Profit'
)
plt.xlabel('Order Index')
plt.ylabel('Profit')
plt.title('First 20 Order Profit Values (Stem Plot)')
plt.legend(loc=1,fontsize=10)
plt.show()
```

Plot:


First 20 Order Profit Values (Stem Plot)
Profit
15000
12500
10000
7500
5000
2500
0
−2500
−5000
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Order Index
Profit

Box Plot

What is a Box Plot?

A Box Plot (also called Box-and-Whisker Plot) is used to show:

- Minimum value
- 25th percentile (Q1)
- Median (Q2)
- 75th percentile (Q3)
- Maximum value
- Outliers

It is mainly used for distribution analysis and outlier detection.

Why do we use Box Plot?

We use it when we want to:

- Understand spread of data
- Detect outliers
- Compare distributions
- Check skewness

It is more powerful than Histogram for identifying outliers.

Data Requirement

At minimum, you need:

- A single numeric column

Example from your dataset:

- Profit
- Sales_Amount
- Quantity

Example::

```
Sales_data = df["Sales_Amount"]

plt.figure(figsize=(5,3))
plt.boxplot(Sales_data)
plt.ylabel("Sales")
plt.title("Sales Distribution")
plt.show()
```

How Box Plot Works

Inside the box plot:

- Bottom line → Minimum (excluding outliers)
- Bottom of box → Q1 (25%)
- Middle line → Median
- Top of box → Q3 (75%)
- Top line → Maximum (excluding outliers)
- Dots outside → Outliers

Important Concept (Very Important)

IQR (Interquartile Range)

$$IQR = Q3 - Q1$$

Outlier rule:

- Below → Q1 − 1.5 × IQR
- Above → Q3 + 1.5 × IQR

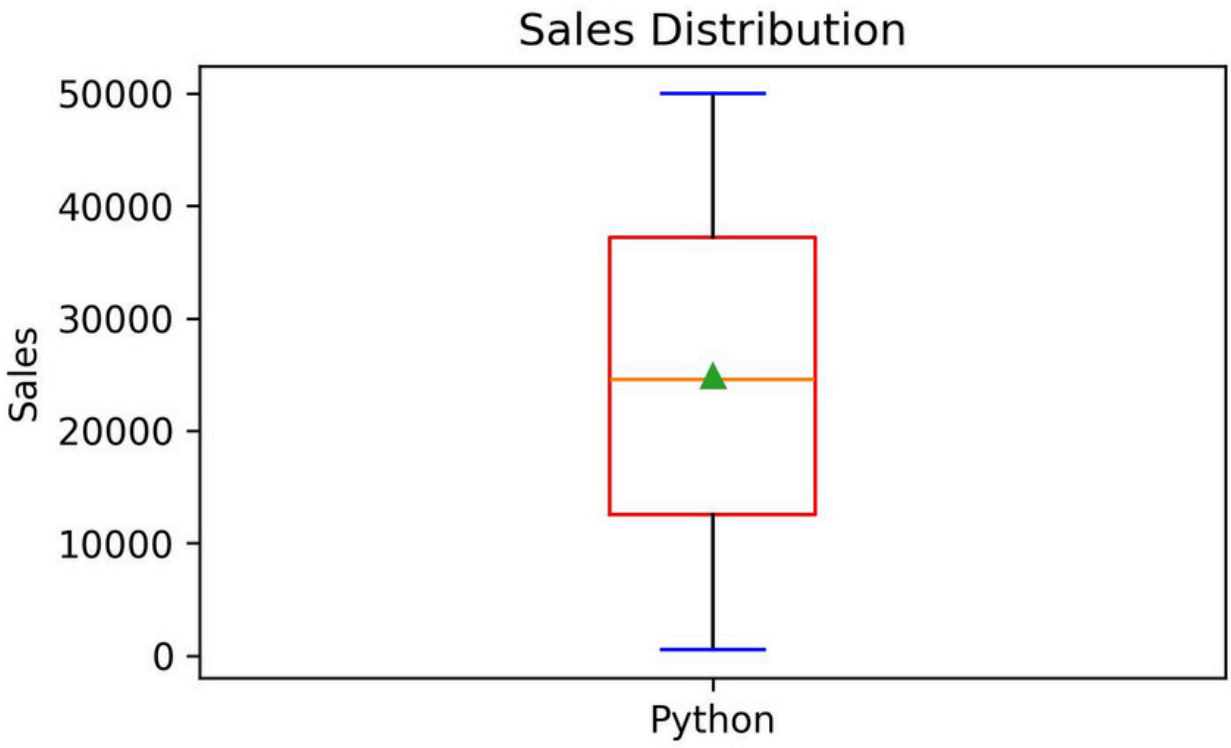
Interviewers love this concept.

When to Use Box Plot

Use it when:

- You want to detect outliers
- Compare multiple numeric columns
- Compare distribution across categories
- Data analyst interviews

```
plt.figure(figsize=(5,3))
plt.boxplot(Sales_data,
widths=0.2,
tick_labels=['Python'],
patch_artist=True,
showmeans=True,
boxprops=dict(color='r'),
capprops=dict(color='b'),
)
plt.ylabel("Sales")
plt.title("Sales Distribution")
plt.show()
```



Area Plot

What is an Area Plot?

An Area Plot is similar to a Line Plot, but the area below the line is filled with color.

It is mainly used to show:

- Trend over time
- Magnitude of change
- Cumulative effect

It visually emphasizes volume.

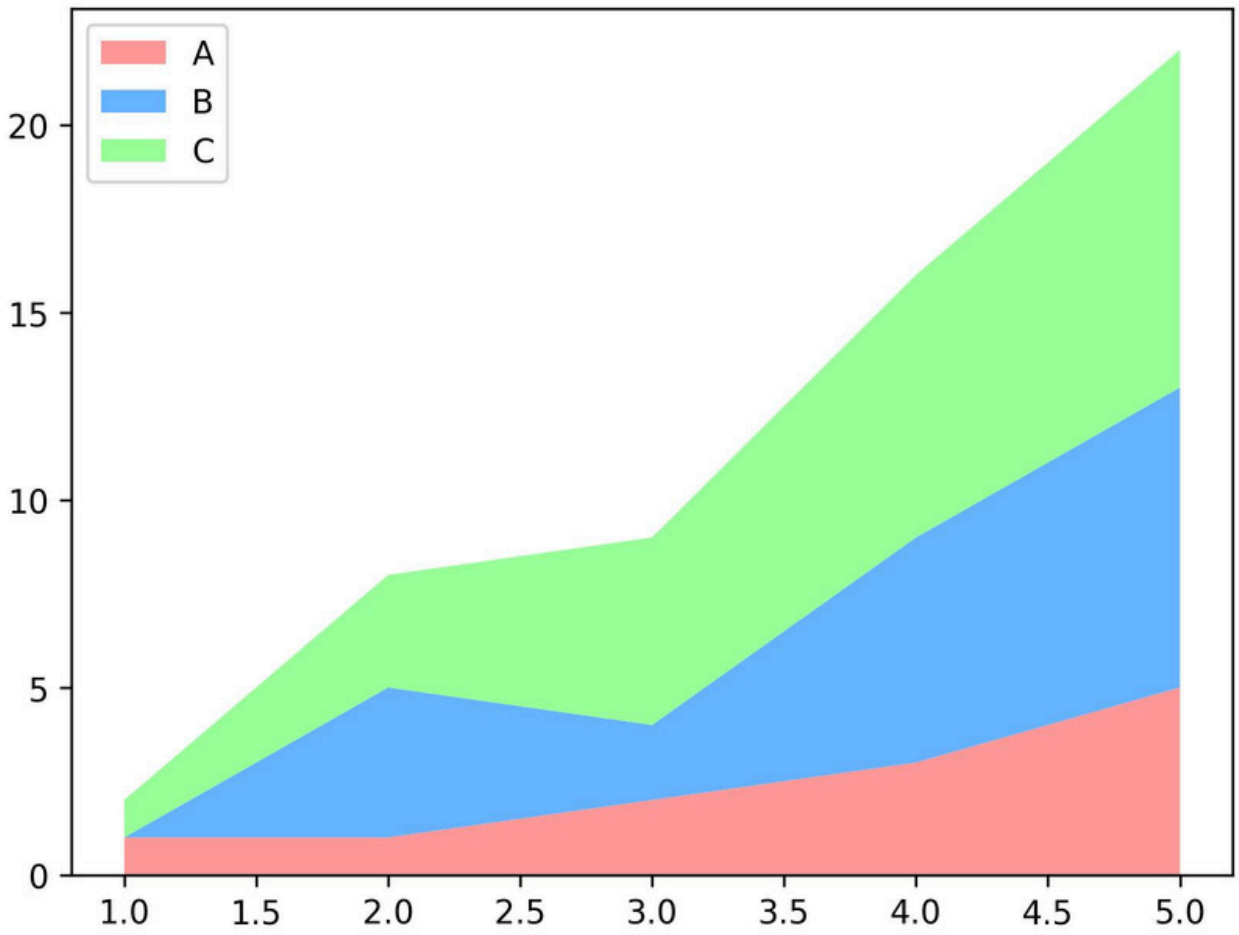
Data Requirement

Minimum requirement:

- X-axis → Sequential data (time, index, month)
- Y-axis → One numeric column

Best used when data has a natural order (time series).

```
y1 = [1, 1, 2, 3, 5]
y2 = [0, 4, 2, 6, 8]
y3 = [1, 3, 5, 7, 9]

plt.stackplot(x, y1, y2, y3, labels=['A', 'B', 'C'],
colors=['#ff9999','#66b3ff','#99ff99'])
plt.legend(loc='upper left')
plt.show()
```



What is a Step Plot?

A Step Plot is similar to a Line Plot, but instead of a straight slanted line between points, it moves in steps:

- Horizontal line
- Then vertical jump
- Then horizontal
- Then vertical

It looks like stairs.

Why Do We Use Step Plot?

We use it when values:

- Change at specific points
- Remain constant until next change
- Represent discrete intervals

It is useful when data does NOT change smoothly.

Data Requirement

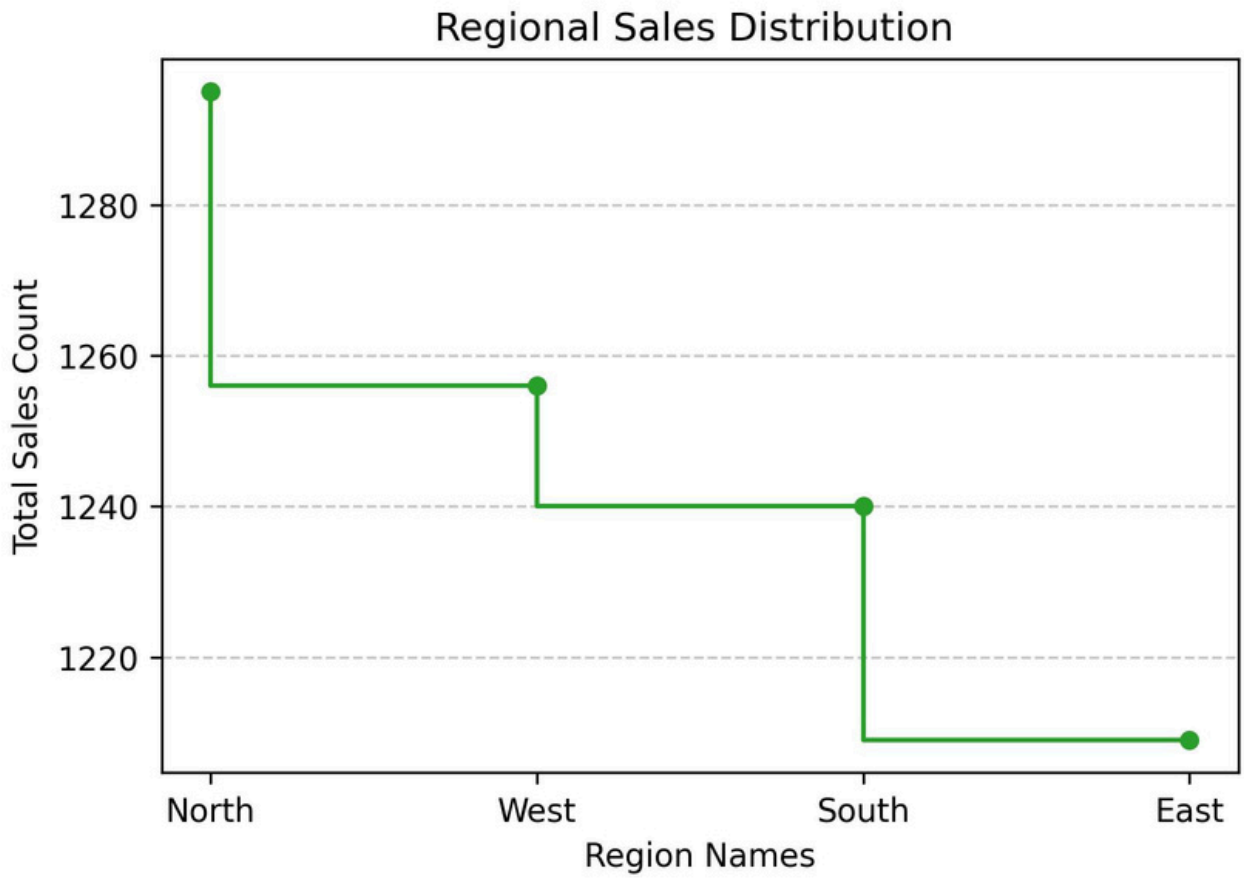
You need:

- X-axis → Ordered values (time, index, month)
- Y-axis → Numeric values

Minimum → 1 numeric column

Querry:

```
plt.figure(figsize=(6,4))
plt.step(
    region_counts.index,
    region_counts.values,
    color='#2ca02c',
    marker='o',ms=5
)
plt.xlabel("Region Names")
plt.ylabel("Total Sales Count")
plt.title("Regional Sales Distribution ")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```


Regional Sales Distribution
Total Sales Count
1280
1260
1240
1220
North
West
South
East
Region Names

Subplot (Matplotlib)

What is a Subplot?

A subplot means:

Multiple plots inside one single figure
Instead of showing charts one by one, you can:

- Show them together
- Compare easily
- Build dashboards

Why do we use Subplots?

We use subplots when:

- We want to compare multiple charts
- We want a clean report/dashboard
- We want everything in one view

This is very common in Data Analyst work.

Basic Syntax (Most Important)

plt.subplot(rows, columns, index)

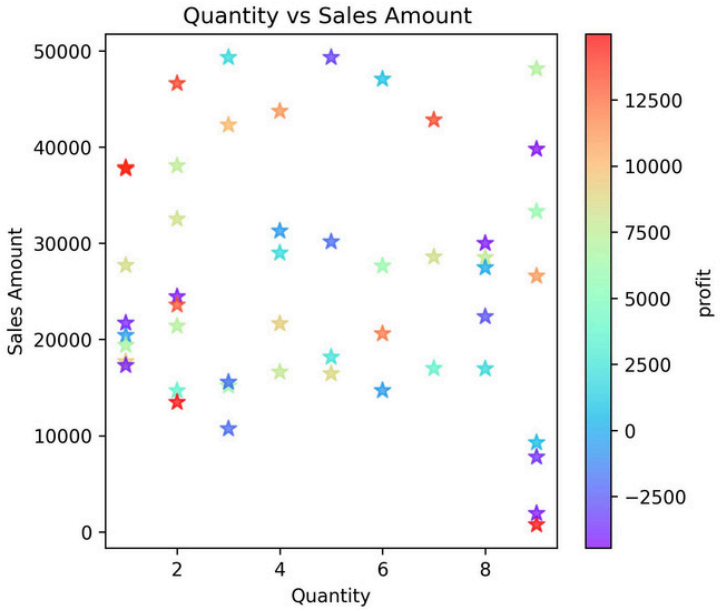
- rows → number of rows
- columns → number of columns
- index → position of the plot (starts from 1)

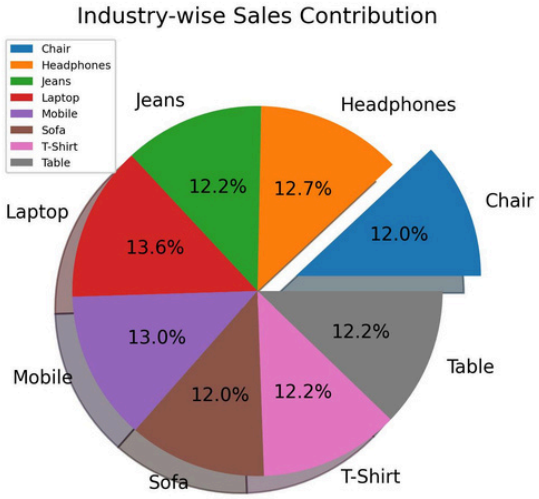
Very Simple Example

```
plt.figure(figsize=(12,11))

# Plot 1
plt.subplot(2, 2, 1)
plt.scatter(
    simple_df["Quantity"],
    simple_df["Sales_Amount"],
    c=simple_df["Profit"],
    cmap="plasma",
    alpha=0.7,
    s= 80, marker="*"
)
plt.colorbar(label="profit")
plt.xlabel("Quantity",fontsize=10)
plt.ylabel("Sales Amount",fontsize=10)
plt.title("Quantity vs Sales Amount ")


# Plot 2
plt.subplot(2, 2, 4)
plt.hist(df["Profit"], bins=10)
plt.title("Profit Distribution")
```

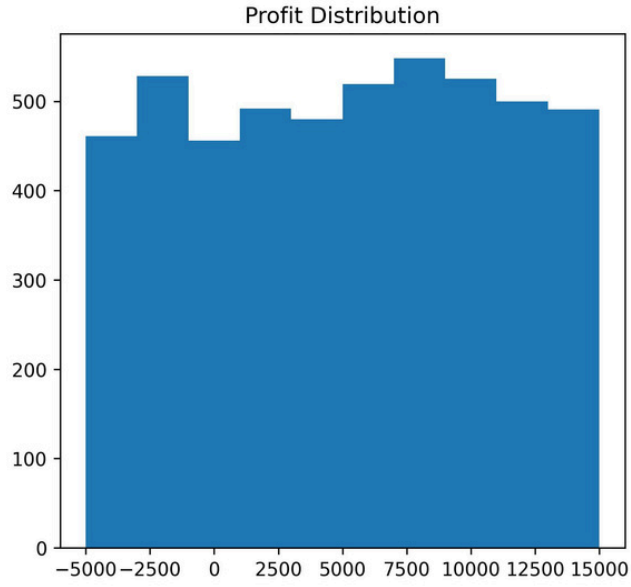
```
#plot3
plt.subplot(2,2,3)
plt.stem(
profit_sample.index,
profit_sample.values,
linefmt=':',
markerfmt='r*',
basefmt='g',
label = 'Profit'
)
plt.xlabel('Order Index')
plt.ylabel('Profit')
plt.title('First 20 Order Profit Values ')
plt.legend(loc=1,fontsize=10)
```

```
#plot 4
plt.subplot(2,2,2)
ex = [0.2, 0, 0, 0,0,0,0,0]
plt.pie(
industry_sales,
labels=industry_sales.index,
explode=ex,
autopct='%1.1f%%',
shadow=True,
radius=0.9,
labeldistance= 1.1,
startangle=0,

)
plt.title('Industry-wise Sales Contribution')
plt.legend(loc=2, fontsize=6)
plt.savefig('subplot.jpg',dpi=300, bbox_inches='tight')

plt.show()
```


Quantity vs Sales Amount
Sales Amount
Quantity
profit
50000
40000
30000
20000
10000
0
2
4
6
8
12500
10000
7500
5000
2500
0
−2500


Industry-wise Sales Contribution
Chair
Headphones
Jeans
Laptop
Mobile
Sofa
T-Shirt
Table
12.2%
12.7%
12.0%
13.6%
13.0%
12.2%
12.0%
12.2%


First 20 Order Profit Values (Stem Plot)
Profit
Order Index
15000
12500
10000
7500
5000
2500
0
−2500
−5000
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Profit Distribution
500
400
300
200
100
0
−5000
−2500
0
2500
5000
7500
10000
12500
15000